/plugins

Plugin management: /plugins list [–verbose] | add [–force] | remove | enable | disable | test | codex [query] [–category] [–refresh] | claude [query] [–category] [–refresh] | stores [–check] | marketplace show | marketplace list | marketplace add [–force] | marketplace remove | help

Overview

`/plugins` manages third-party plugins written for OpenAI Codex and Claude Code/Desktop. Plugin JavaScript runs on an embedded, budget-sandboxed engine (`rquickjs`) inside the ragent binary - no external Node.js, Deno, or Bun installation is required. Enabled plugins contribute tools (registered as `plugin_<id>_<tool>`) and slash commands through a versioned `ragent` host API.

A bare `/plugins`, `/plugins help`, and any unrecognised subcommand all print the same usage block and create or modify no files.

The same operations are available from a shell via the `ragent plugins <sub>` CLI parity surface; the two surfaces share one help text and one set of command handlers.

Syntax

```
/plugins list [--verbose]      # list discovered plugins
/plugins list --mcp            # live MCP tool inventory (servers, status, tools)
/plugins add <source> [--force]  # install a plugin (enabled on install)
/plugins remove <pluginid>     # uninstall (refused while enabled)
/plugins enable <pluginid>     # enable, load, register tools/commands
/plugins disable <pluginid>    # unload, deregister tools/commands
/plugins test <pluginid>       # isolated harness: load + invoke each tool once
/plugins codex [query] [--category <name>] [--refresh]    # browse the Codex marketplace
/plugins claude [query] [--category <name>] [--refresh]   # browse the Claude marketplace
/plugins stores [--check]      # report each store's endpoint and provenance
/plugins marketplace show      # list the configured marketplaces (both stores) in one name/url/d
/plugins marketplace claude list                          # list one store's marketplaces
/plugins marketplace claude add anthropics/claude-plugins-community  # add a marketplace
/plugins marketplace claude add owner/repo --force        # replace an existing marketplace
/plugins marketplace claude remove claude-plugins-community  # remove a marketplace
/plugins help                  # usage block
```

Options / Subcommands

Form	Description
`/plugins list [--verbose]`	Open the interactive plugin panel. The table is pinned at the top - a fixed column heading and separator above a scrollable block-cursor list - and the capability blocks (contributions, unsupported capabilities, errors, and a totals summary) are listed at the end of the same scrollable body. Each table row shows id, name, version, dialect, state (`disabled`/`enabled`/`loaded`/`errored`), contributed tool/command/skill/agent/hook counts, and the `MCP` server / `MCP Tools` tool counts. An enabled plugin's row is tinted yellow. `Up`/`Down` (or `k`/`j`) move the cursor, `Home`/`End` jump to the ends, `Space` toggles the highlighted plugin's enabled state (enable/disable) and shows the outcome on the panel's feedback line, and `Esc` closes the panel. `--verbose` (or `-v`) appends the per-plugin telemetry block. The panel shares one discovery path with the `ragent plugins list` CLI, so the two cannot disagree.
`/plugins list --mcp`	Print the live MCP tool inventory: every connected MCP server (including those bridged from a plugin's `mcpServers` section) with its status, tool count, and each tool's registry name (`<tool> -> mcp_<server>_<tool>`).
`/plugins add <source> [--force]`	Install a plugin and validate its manifest; reports the plugin id and dialect. The plugin is recorded enabled and loads at the next session start; turn it off with `/plugins disable`. Refuses a duplicate id unless `--force` is given.
`/plugins remove <pluginid>`	Uninstall a plugin from the store. Refused while the plugin is enabled.
`/plugins enable <pluginid>`	Mark the plugin enabled, load it into the current session, and register its tools and commands. Reports the declared permissions and the load outcome.
`/plugins disable <pluginid>`	Mark the plugin disabled, unload it, and deregister every tool and command it contributed, confirming how many of each were removed. Files are not deleted.
`/plugins test <pluginid>`	Load the plugin in an isolated harness (never touching the live session), invoke each contributed tool once with schema-derived sample arguments, and report per-step `[ ok ]`/`[fail]` results with wall-clock time.

Form	Description
`/plugins codex [query] [--category <name>] [--refresh]`	Open the interactive Codex marketplace browser over the store index: `query` pre-fills the search field, `--category` pre-selects a left-hand category row, and `--refresh` bypasses on-disk store cache. The left-hand navigator lists `ALL`, then `Installed` (the plugins already in the store scan, matched by id; `--category installed` pre-selects it), plus the distinct categories the index declares (`Tab` focus, `Up`/`Down` move, `Enter` apply, `c` clear, or click/scroll with the mouse). Result rows are sorted alphabetically by plugin id, and a query or category filter narrows the already-sorted set.
`/plugins claude [query] [--category <name>] [--refresh]`	Open the interactive Claude marketplace browser, with the same `query`, `--category`, and `--refresh` arguments (and the same alphabetical result ordering).
`/plugins stores [--check]`	Report each store's effective endpoint and its source; `--check` also contacts each store and reports availability and plugin count.
`/plugins marketplace show`	List the marketplaces configured for both stores: one section per store (Codex, Claude), each a table with one row per marketplace showing name, url, and description (store label plus `config`/`default`/`added` provenance). A pure config read - no network.
`/plugins marketplace <store> list`	List one store's configured marketplaces (the base endpoint plus every added marketplace, tagged `default`/`config`/`added`). `<store>` is `codex` or `claude`.
`/plugins marketplace <store> add <source> [--force]`	Add a marketplace to a store. `<source>` is an `owner/repo` GitHub slug, a `git@github.com:owner/repo.git` SSH remote, an `https://` marketplace-index URL, or a local marketplace directory. GitHub forms resolve to that store's raw marketplace document on branch `main`. The marketplace is written to your **user-global** config (`plugins.stores.marketplaces`) and used by the next `/plugins <store>` browse; `--force` replaces an existing marketplace with the same name.
`/plugins marketplace <store> remove <name>`	Remove an added marketplace from a store by name.
`/plugins help`	Print the usage block. A bare `/plugins` or an unknown subcommand does the same.

Adding marketplaces

`/plugins codex` and `/plugins claude` browse the union of the store's base endpoint and every marketplace added for that store, so a community or private catalogue appears alongside the official one in the same panel (entries are de-duplicated by plugin id):

```
/plugins marketplace claude add anthropics/claude-plugins-community
/plugins marketplace claude add https://example.org/internal/marketplace.json
/plugins marketplace claude add ./my-marketplace
/plugins claude                 # now lists both the official and the added marketplaces
/plugins marketplace claude list
/plugins marketplace claude remove claude-plugins-community
```

Plugin details (version and description)

A vendor marketplace is a listing and often omits the plugin's version, and the Codex catalogue omits a description as well. After the index loads, the browser fetches each plugin's own manifest to fill those in, showing a `loading details...` hint in the title while it works. A version that stays unknown (for example a plugin whose source is an archive with no GitHub repository to read a manifest from) renders as – rather than a blank cell. No plugin code is executed to read a manifest.

Sources accepted by `/plugins add`

- a local directory containing a plugin manifest;
- a local `.zip` or `.tar.gz` package file;
- an `https://` URL pointing at a `.zip`/`.tar.gz` package (non-`https` URLs are refused).

Examples

List everything with telemetry:

```
/plugins list --verbose
```

Install a plugin from a local directory and enable it:

```
/plugins add ./my-plugin
/plugins enable my-plugin
```

Test a plugin without touching the live session:

```
/plugins test my-plugin
```

Unload without deleting files:

```
/plugins disable my-plugin
```

Output

Reports are prefixed with a `From: /plugins <sub>` attribution line. `list` renders a fixed-width table with one count column per contribution kind (`Tools`, `Commands`, `Skills`, `Agents`, `Hooks`) plus two MCP columns - `MCP` (the number of MCP servers the plugin declares) and `MCP Tools` (the total tools those servers advertise, summed from the live MCP client; `?` when a server has not connected, never `0`):

```
| ID |Name |Version |Dialect |State    |Tools|Commands|Skills|Agents|Hooks|MCP|MCP Tools|
```

followed by `Contributions:`, `Unsupported capabilities:`, and `Errors:` sections — the `Contributions:` block lists the detailed names for each kind, e.g. `skills [db-setup]; agents [agents/security-reviewer.md]; hooks [PreToolUse, SessionStart]; mcp [mongodb.mongodb (35 tools)] (1 server(s), 35 tool(s))` — and a summary line (`Total: N plugin(s) - X enabled, X disabled, X errored`, with loaded plugins counted as enabled).

`test` renders one line per harness step:

```
[ ok ] discovery (2 ms)
[ ok ] manifest (0 ms)
[ ok ] entry (5 ms)
[ ok ] tool ping (1 ms)
```

and ends by confirming the harness was unloaded with the live session untouched.

Configuration

The `plugins` block in `ragent.json` controls the subsystem (`enabled`, `max_execution_ms`, `max_entry_ms`, `max_memory_mb`, `store_dir`, and per-plugin `permissions`). With `plugins.enabled: false`, every subcommand other than `help` reports the subsystem is disabled and no plugin code executes. Plugins are discovered under `.ragent/plugins/` (project), falling back to `~/.config/ragent/plugins/` (user-global).

See `docs/howtos/config.md` §7.35 and `specs/plugins/SPEC.md`.

Related

- `docs/howtos/plugins.md` - the full plugin-system manual
- `/reload` - reload customizations after plugin files change
- `/tools` - list every registered tool, plugin tools included
- `ragent plugins <sub>` - CLI parity for the same subcommands
- `specs/plugins/SPEC.md` - the full plugin-system specification